Neural Architecture Search via Algoritmo Genetico

Documento di studio per l'orale

Esame: Elementi di Intelligenza Artificiale (6 CFU)

Corso di laurea in Ingegneria Informatica

Cosa fa il progetto in una frase

Un Algoritmo Genetico cerca automaticamente la migliore architettura di una rete neurale (numero di layer, neuroni per layer, funzione di attivazione) per un dato dataset, valutando ogni architettura tramite l'accuratezza ottenuta dopo training con backpropagation, penalizzata per la complessità della rete.

12 maggio 2026

Indice

1 Introduzione **4**
1.1 L'idea del progetto in 60 secondi . . . 4
1.2 Cos'è il Neural Architecture Search (NAS) . . . 4
1.3 Originalità e rilevanza . . . 4

2 Background di Machine Learning **6**
2.1 Apprendimento supervisionato . . . 6
2.2 Train, Validation e Test set . . . 6
2.2.1 Holdout vs k-fold cross validation . . . 6
2.2.2 Stratificazione . . . 7
2.3 Classificazione vs Regressione . . . 7
2.4 Iperparametri vs parametri . . . 7
2.5 Overfitting e generalizzazione . . . 7

3 Reti Neurali Artificiali **9**
3.1 Dal neurone biologico al neurone artificiale . . . 9
3.2 Topologia: input, hidden, output . . . 9
3.3 Funzioni di attivazione . . . 9
3.4 Forward step . . . 10
3.5 Loss functions . . . 10
3.6 Backward step e Gradient Descent . . . 11
3.6.1 L'intuizione . . . 11
3.6.2 Backpropagation: l'idea . . . 11
3.6.3 SGD vs batch vs mini-batch . . . 12
3.7 Inizializzazione dei pesi . . . 12

4 Algoritmi Genetici **14**
4.1 La metafora biologica . . . 14
4.2 Componenti fondamentali . . . 14
4.3 Il ciclo evolutivo . . . 14
4.4 Selezione: torneo . . . 15
4.5 Crossover . . . 16
4.6 Mutazione . . . 16
4.6.1 I tre tipi di mutazione del progetto . . . 16
4.7 Elitismo . . . 16
4.8 GA come variante della ricerca locale . . . 17

5 Implementazione del progetto **18**
5.1 Architettura del sistema . . . 18
5.2 Rappresentazione del cromosoma . . . 18
5.3 Funzione di fitness . . . 19
5.3.1 Perché penalizzare la complessità . . . 19
5.3.2 Perché logaritmica e non lineare . . . 19
5.3.3 Perché K valutazioni . . . 20

5.4 Operatori genetici (pseudocodice) . . . 21
5.4.1 Selezione a torneo . . . 21
5.4.2 Crossover a doppio punto di taglio . . . 21
5.4.3 Mutazione . . . 21
5.5 Il ciclo principale (run) . . . 22
5.6 Preprocessing del dataset . . . 22
5.6.1 Split train / validation / test . . . 22
5.6.2 Normalizzazione (StandardScaler) . . . 22
5.6.3 One-hot encoding . . . 23
5.7 La baseline di confronto . . . 23

6 Concetti chiave trasversali **25**
6.1 Overfitting nel contesto del progetto . . . 25
6.2 Data leakage . . . 25
6.3 Esplorazione vs sfruttamento . . . 26
6.4 Generalizzabilità: metodo vs risultato . . . 26
6.5 Ottimizzazione annidata: due livelli . . . 27

7 Esperimenti e Risultati **28**
7.1 Metodologia: analisi di sensibilità . . . 28
7.1.1 Dataset principale: `load_digits` . . . 28
7.1.2 Configurazione di base . . . 28
7.2 Test 1 — Numero di epoche . . . 29
7.3 Test 2 — Numero di valutazioni K . . . 29
7.4 Test 3 — Learning rate . . . 30
7.5 Test 4 — Penalità di complessità λ . . . 30
7.6 Test 5 — Mutation rate . . . 31
7.7 Test 6 — Dimensione della popolazione . . . 32
7.8 Test 7 — Numero di generazioni . . . 32
7.9 Test 8 — Generalizzabilità su dataset diversi . . . 33
7.10 Test 9A — Bias ottimistico val vs test . . . 34
7.11 Test 9B — Leakage da preprocessing . . . 34
7.12 Riepilogo della configurazione ottimale . . . 35

8 Discussione e Conclusioni **37**
8.1 Cosa il progetto dimostra . . . 37
8.2 Limiti del progetto . . . 37
8.3 Sviluppi futuri . . . 37

9 Domande tipo orale (con risposte ragionate) **39**
9.1 Sul progetto in generale . . . 39
9.2 Su reti neurali . . . 39
9.3 Su algoritmi genetici . . . 40
9.4 Su NAS e fitness . . . 41
9.5 Su validazione e leakage . . . 42
9.6 Domande di approfondimento . . . 42

A Glossario dei termini chiave **44**

1 Introduzione

1.1 L'idea del progetto in 60 secondi

Quando si progetta una rete neurale, la scelta dell'architettura (*quanti layer? quanti neuroni? quale funzione di attivazione?*) è quasi sempre fatta a mano, per tentativi ed errori, basandosi sull'esperienza del progettista. Il problema è che lo spazio delle architetture possibili è enorme: anche limitandosi a $1 \ldots 5$ layer, $4 \ldots 64$ neuroni per layer e 5 funzioni di attivazione, si ottengono milioni di combinazioni. Provarle tutte è impensabile.

L'idea del progetto è semplice e potente: invece di cercare a mano, **lasciamo cercare a un Algoritmo Genetico (GA)**. Il GA imita l'evoluzione naturale: parte da una popolazione casuale di architetture, seleziona le migliori, le combina (crossover), introduce piccole variazioni (mutazione) e ripete il processo per molte generazioni. Quello che emerge alla fine è un'architettura quasi ottima, scoperta automaticamente.

I pesi della rete vengono comunque imparati con **backpropagation classica**: il GA decide solo la *forma* della rete, la backpropagation ne riempie i *valori*. Si tratta quindi di un'**ottimizzazione annidata**: due ottimizzatori operano contemporaneamente su scale diverse.

1.2 Cos'è il Neural Architecture Search (NAS)

Il **Neural Architecture Search** è il sottocampo dell'*automated machine learning* che studia come automatizzare la progettazione delle reti neurali. È strettamente collegato a:

- **Ottimizzazione degli iperparametri** — gli iperparametri sono tutti i parametri di un modello che *non* vengono appresi dai dati (learning rate, topologia, dimensione). Tradizionalmente si scelgono con *grid search*, *random search* o intuizione. NAS lo fa con tecniche evolutive o di reinforcement learning.
- **Meta-apprendimento** — "imparare ad apprendere". Un vero sistema di meta-learning estrae *principi generali* validi su molti dataset (es. "problemi con poche classi e feature continue beneficiano di pochi layer ReLU"). Questo progetto è più correttamente *architecture search* per un singolo dataset, non meta-learning in senso stretto.

NAS è un campo industrialmente rilevante: modelli come *EfficientNet* (Google) sono nati da ricerca automatica dell'architettura, e NAS è la tecnica di riferimento quando bisogna deployare reti su hardware con risorse limitate (mobile, IoT).

1.3 Originalità e rilevanza

Domanda tipo orale

"Il tuo progetto è originale?"
No, nel senso accademico stretto: la NAS via GA è una tecnica nota dagli anni '90. Ciò che è originale è la *combinazione specifica* qui realizzata:

- rete neurale scritta da zero con backpropagation manuale (solo `numpy`, niente TensorFlow/PyTorch);
- GA scritto da zero che ottimizza l'architettura;

- penalità di complessità logaritmica per evitare reti grandi inutili;
- valutazione multipla ($K = 3$) per ridurre la varianza dovuta all'inizializzazione casuale dei pesi.

Tutti i componenti sono costruiti senza librerie di machine learning preconfezionate: questo dimostra la comprensione *sotto il cofano* dei due paradigmi — reti neurali e algoritmi evolutivi — più che la capacità di usare API.

Punti chiave da ricordare

- Il progetto risolve il problema della scelta manuale dell'architettura di una rete neurale.
- Usa un GA per esplorare lo spazio delle architetture e la backpropagation per addestrare ogni rete candidata.
- È un caso di *ottimizzazione annidata*: due livelli di ottimizzazione, GA sopra e backpropagation sotto.
- Il valore non è nell'originalità del metodo, ma nell'implementazione completa da zero di entrambi i paradigmi.

2 Background di Machine Learning

Per inquadrare il progetto serve un nucleo solido di concetti di machine learning. Tutto ciò che segue verrà richiamato nel resto del documento.

2.1 Apprendimento supervisionato

Il **learning from labels** (apprendimento supervisionato) è il paradigma in cui ogni esempio del dataset è una coppia (x, y) dove x è un vettore di feature e y è l'etichetta vera (la "risposta corretta"). L'obiettivo del modello è imparare una funzione f tale che $f(x) \approx y$ per esempi mai visti prima.

Dopo l'addestramento il modello deve *generalizzare*: dare la risposta giusta su input nuovi. Il modo più semplice per misurarlo è separare i dati in due (o tre) sottoinsiemi disgiunti.

2.2 Train, Validation e Test set

- **Training set** (tipicamente 60–80% dei dati). Su questo il modello aggiusta i suoi parametri — in una rete neurale, i pesi vengono modificati dalla backpropagation.
- **Validation set** (10–20%). Non viene usato per aggiornare i pesi, ma per scegliere fra modelli/configurazioni (es. fra architetture diverse, learning rate diversi). Nel nostro progetto è il set su cui il GA misura la fitness di ogni individuo.
- **Test set** (10–20%). Toccato solo alla fine, per stimare onestamente quanto il modello generalizza. Se lo si usa durante la ricerca, si introduce **data leakage**: il modello impara a fare bene su dati che dovrebbero essere segreti.

Concetto chiave: separazione delle responsabilità tra i set

Train, validation e test svolgono tre ruoli *diversi e non intercambiabili*:

- Il train serve a *imparare*.
- Il validation serve a *decidere*.
- Il test serve a *misurare* (una volta sola).

Confondere questi ruoli porta a stime gonfiate delle prestazioni.

2.2.1 Holdout vs k-fold cross validation

Holdout è il metodo più semplice: si divide il dataset una volta sola in train/val/test. Veloce, ma il risultato dipende da come è stata fatta la divisione. Funziona bene quando il dataset è abbastanza grande da essere rappresentativo in ogni split.

K-fold cross validation divide il dataset in k pezzi (*fold*) di uguale dimensione, addestra k volte usando ogni volta un fold diverso come validation, e media i risultati. Stima più affidabile, ma costa k volte di più. In questo progetto si usa l'holdout (sufficiente data la mole dei dataset utilizzati e il costo già alto del GA).

2.2.2 Stratificazione

Quando si fa lo split è importante mantenere le proporzioni delle classi in ciascun sottoinsieme. Senza stratificazione, su dataset piccoli può capitare che una classe finisca quasi tutta nel training set e quasi niente nel validation set — il modello sembrerà funzionare benissimo ma in realtà non ha mai visto certi casi.

2.3 Classificazione vs Regressione

Nel supervisionato si distinguono due tipi di problemi a seconda di cosa è y:

- **Classificazione** — y è una categoria discreta. *"Questo fiore è setosa, versicolor o virginica?"*. La metrica naturale è l'*accuracy* (% di predizioni corrette). Sull'output si usa tipicamente `softmax` per ottenere una distribuzione di probabilità sulle classi.
- **Regressione** — y è un valore continuo. *"Quanto costerà questa casa?"*. La metrica è MSE/RMSE (errore quadratico medio). Sull'output si usa una funzione lineare.

Il progetto è interamente di *classificazione*: tutti e quattro i dataset di test (Iris, Wine, Breast Cancer, Digits) chiedono di assegnare ogni esempio a una categoria.

2.4 Iperparametri vs parametri

- I **parametri** di un modello sono i valori che il modello *impara dai dati* durante l'addestramento — in una rete neurale, i pesi e i bias.
- Gli **iperparametri** sono le scelte di configurazione che fissiamo *prima* dell'addestramento: learning rate, numero di epoche, dimensione della popolazione del GA, mutation rate, λ della penalità di complessità, ecc. Anche l'*architettura* stessa è un iperparametro — ed è esattamente quella che il GA cerca di scegliere automaticamente.

Concetto chiave: la struttura tipica di un esperimento di ML

1. Si fissano gli iperparametri (a mano o tramite ricerca automatica).
2. Si addestra il modello sul training set facendo scendere il gradiente della loss.
3. Si misurano le prestazioni sul validation set.
4. Si ripetono i passi 1–3 con configurazioni diverse e si sceglie la migliore.
5. Si valuta una sola volta la configurazione vincente sul test set per riportare il risultato finale.

2.5 Overfitting e generalizzazione

Un modello in **overfitting** ha imparato a memoria i dati di training invece di estrarne le regole generali. Si manifesta con un *accuracy alta sul training set ma bassa sul validation/test set*. Le cause più comuni:

- modello troppo grande rispetto alla quantità di dati (troppi parametri liberi);
- training troppo lungo (la rete si specializza sui dettagli);
- pattern accidentali nei dati di training (rumore).

L'**underfitting** è il problema opposto: il modello è troppo semplice per cogliere la struttura del problema — accuracy bassa sia sul training che sul validation.

Concetto chiave: bias/variance trade-off

Bias alto / variance bassa = underfitting: il modello fa errori sistematici perché è troppo rigido. **Bias basso / variance alta** = overfitting: il modello cambia molto al variare dei dati e impara il rumore. La sweet spot è in mezzo, e spesso la si raggiunge regolarizzando il modello — nel nostro caso con la penalità λ sulla complessità, che incentiva architetture più snelle.

Punti chiave da ricordare

- Il dataset si divide in train (impara), validation (decide), test (misura una sola volta).
- I parametri si imparano dai dati; gli iperparametri si fissano prima del training.
- L'architettura è un iperparametro — ed è quello che il GA cerca automaticamente.
- L'overfitting si combatte con dataset adeguati, regolarizzazione o vincoli sulla complessità.

3 Reti Neurali Artificiali

3.1 Dal neurone biologico al neurone artificiale

Una rete neurale artificiale è un modello computazionale ispirato al funzionamento del cervello biologico. L'unità elementare è il **neurone**: riceve segnali in ingresso, li combina linearmente tramite *pesi* (le "sinapsi"), aggiunge un *bias* e applica una funzione di attivazione non lineare per produrre l'uscita.

Per un singolo neurone i:

$$a_i = f(\boldsymbol{w}_i^\top \boldsymbol{x} + b_i) \tag{1}$$

dove $\boldsymbol{w}_i$ è il vettore dei pesi, $\boldsymbol{x}$ è il vettore di input, b_i il bias, f la funzione di attivazione e a_i l'uscita (*attivazione*).

Per un intero **layer** di neuroni si raggruppa in forma matriciale:

$$\boldsymbol{a} = f(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}) \tag{2}$$

con $\boldsymbol{W}$ matrice dei pesi (una riga per neurone) e $\boldsymbol{b}$ vettore dei bias.

3.2 Topologia: input, hidden, output

Una rete *feed-forward* (l'unica usata in questo progetto) è strutturata in tre tipi di layer in cascata:

- **Layer di input** — riceve le feature del dataset, una per neurone. Non applica nessuna trasformazione, è un puro punto di ingresso.
- **Hidden layer(s)** — uno o più layer intermedi. Ogni layer trasforma la rappresentazione dei dati applicando $\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$ seguito da una funzione di attivazione non lineare. È qui che la rete impara feature di livello superiore.
- **Layer di output** — produce la predizione finale. La funzione di attivazione dipende dal problema: `softmax` per classificazione multiclasse, `sigmoid` per classificazione binaria, lineare per regressione.

Perché servono le funzioni di attivazione non lineari? Senza di esse una rete con N layer è matematicamente equivalente a una rete con 1 layer: la composizione di trasformazioni lineari è lineare. La non linearità è ciò che permette alla rete di rappresentare funzioni arbitrariamente complesse (*universal approximation theorem*).

3.3 Funzioni di attivazione

ReLU (Rectified Linear Unit) — $f(x) = \max(0, x)$. Veloce, derivata semplice (1 se $x > 0$, 0 altrimenti). Funziona molto bene nei layer nascosti perché non satura per $x > 0$. Problema: *dying ReLU* — se un neurone riceve sempre input negativi smette di apprendere (gradiente sempre 0).

Leaky ReLU — $f(x) = x$ se $x > 0$, altrimenti αx con α piccolo (es. 0.01). Risolve il dying ReLU mantenendo un piccolo gradiente per $x < 0$.

Sigmoid — $f(x) = \frac{1}{1+e^{-x}}$. Output in $(0, 1)$. Usata storicamente nei layer nascosti, oggi sconsigliata perché *satura* (per $|x|$ grande la derivata si avvicina a zero, causando il *vanishing gradient*). Resta utile come attivazione di output per problemi binari.

Linear — $f(x) = x$. Identità. Usata in output per regressione, raramente nei layer nascosti.

Softmax — $f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$. Trasforma un vettore in una distribuzione di probabilità (somma a 1). È la scelta standard per l'output di una classificazione multiclasse: il neurone con probabilità più alta indica la classe predetta.

Concetto chiave: funzioni di attivazione e GA

Nel progetto le funzioni di attivazione disponibili sono ReLU, Leaky ReLU, Sigmoid, Linear e Softmax. Per i layer nascosti il GA può scegliere liberamente fra le prime quattro; sull'output la `softmax` è fissa (tutti i task sono di classificazione multiclasse). La possibilità di mescolare attivazioni diverse fra layer è una scelta progettuale: il GA può combinare ReLU al primo layer e Leaky ReLU al secondo, ad esempio.

3.4 Forward step

Il **forward step** (o *forward pass*) propaga un input attraverso la rete per produrre la predizione. Per una rete con L hidden layer:

$$\boldsymbol{a}^{(0)} = \boldsymbol{x}, \qquad \boldsymbol{a}^{(\ell)} = f^{(\ell)}\Big(\boldsymbol{W}^{(\ell)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell)}\Big) \quad \ell = 1, \ldots, L+1 \tag{3}$$

L'output finale $\hat{\boldsymbol{y}} = \boldsymbol{a}^{(L+1)}$ è la predizione della rete. Durante il training si confronta $\hat{\boldsymbol{y}}$ con l'etichetta vera $\boldsymbol{y}$ e si calcola la *loss* (errore).

3.5 Loss functions

La **funzione di loss** misura quanto la predizione si discosta dalla verità. La scelta dipende dal problema:

- **Mean Squared Error (MSE)** — per regressione:

$$\mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$$

- **Binary Cross-Entropy** — per classificazione binaria:

$$\mathcal{L} = -\big(y \log \hat{y} + (1-y)\log(1-\hat{y})\big)$$

- **Categorical Cross-Entropy** — per classificazione multiclasse (l'unica usata in questo progetto):

$$\mathcal{L} = -\sum_i y_i \log \hat{y}_i$$

  dove $\boldsymbol{y}$ è in *one-hot encoding* e $\hat{\boldsymbol{y}}$ è l'uscita della softmax.

Concetto chiave: softmax + cross-entropy: la coppia che semplifica i conti

Quando l'output è softmax e la loss è categorical cross-entropy, il gradiente combinato si scrive in modo molto compatto:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{z}^{(L+1)}} = \hat{\boldsymbol{y}} - \boldsymbol{y}$$

dove $\boldsymbol{z}^{(L+1)}$ è il pre-attivazione dell'output. Questo risultato è sfruttato nel codice per evitare di calcolare separatamente la derivata della softmax (che è una matrice jacobiana) e quella della cross-entropy.

3.6 Backward step e Gradient Descent

Il **backward step** (o *backpropagation*) è l'algoritmo che calcola, per ogni peso w della rete, la derivata parziale $\frac{\partial \mathcal{L}}{\partial w}$ (il *gradiente*). Una volta ottenuto, si aggiornano i pesi nella direzione opposta al gradiente:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w} \tag{4}$$

dove η è il **learning rate**. Questa è la *discesa del gradiente* (gradient descent).

3.6.1 L'intuizione

Il gradiente è la pendenza della funzione di loss rispetto al peso. Spostarsi nella direzione opposta significa scendere lungo la pendenza, verso il minimo della loss. Il learning rate η controlla la *dimensione del passo*:

- η troppo *piccolo* $\rightarrow$ la convergenza è lentissima, in N epoche fissate la rete non arriva al minimo.
- η troppo *grande* $\rightarrow$ i passi sono così ampi che la rete "salta" oltre il minimo, oscilla o diverge.
- η *giusto* $\rightarrow$ discesa rapida e stabile fino al minimo.

3.6.2 Backpropagation: l'idea

Il problema di calcolare $\frac{\partial \mathcal{L}}{\partial w}$ per *tutti* i pesi della rete è che la loss dipende dal peso w in modo indiretto: w contribuisce all'attivazione di un neurone, che contribuisce a quella del layer successivo, e così via fino all'output. Bisogna quindi applicare la **regola della catena** ripetutamente.

L'osservazione chiave: i calcoli del layer ℓ riusano risultati del layer $\ell + 1$. Se si parte dall'output e si va all'indietro layer per layer, si calcola tutto in un singolo *forward pass* seguito da un singolo *backward pass* — ecco perché si chiama *back-propagation*.

Algoritmo 1 Backpropagation (schema concettuale)

Input: input $\boldsymbol{x}$, etichetta $\boldsymbol{y}$, rete con L hidden layer

Output: aggiornamento di tutti i pesi $\boldsymbol{W}^{(\ell)}$ e bias $\boldsymbol{b}^{(\ell)}$

```
Forward: calcola a^(ℓ) per ℓ = 1, ..., L+1 e memorizza tutto
Calcola la loss L(ŷ, y)
Backward: calcola δ^(L+1) = ∇_{z^(L+1)} L
for ℓ = L downto 1 do
    δ^(ℓ) ← (W^(ℓ+1))^⊤ δ^(ℓ+1) ⊙ f'^(ℓ)(z^(ℓ))
end for
for ℓ = 1 to L+1 do
    W^(ℓ) ← W^(ℓ) − η δ^(ℓ) (a^(ℓ−1))^⊤
    b^(ℓ) ← b^(ℓ) − η δ^(ℓ)
end for
```

Le quantità $\boldsymbol{\delta}^{(\ell)}$ sono il "gradiente locale" di ogni layer, calcolato in modo ricorsivo dall'output verso l'input.

3.6.3 SGD vs batch vs mini-batch

Esistono tre modalità di gradient descent a seconda di quanti esempi si usano per stimare il gradiente prima di aggiornare i pesi:

- **Batch GD** — usa tutti gli esempi del training set per ogni aggiornamento. Stima accurata, ma lenta e costosa.
- **Stochastic GD (SGD)** — usa un solo esempio alla volta. Stima rumorosa ma il rumore stesso aiuta a uscire da minimi locali. È la modalità usata in questo progetto.
- **Mini-batch GD** — compromesso: $32, 64, 128$ esempi alla volta. Standard nei framework moderni.

3.7 Inizializzazione dei pesi

I pesi non possono essere inizializzati a zero (tutti i neuroni diventerebbero identici e imparerebbero la stessa cosa — problema della *symmetry breaking*) né a valori troppo grandi (gradient explosion). Le strategie più usate sono:

- **Inizializzazione di He** — pesi $\sim \mathcal{N}(0, \sigma^2)$ con $\sigma^2 = 2/n_{\text{in}}$. Ottimizzata per ReLU/Leaky ReLU.
- **Inizializzazione di Xavier (o Glorot)** — $\sigma^2 = 1/n_{\text{in}}$. Ottimizzata per sigmoid e tanh.

In questo progetto si applica He per ReLU e Leaky ReLU, Xavier per le altre funzioni. È un dettaglio importante: senza inizializzazione ragionata, reti più profonde tendono al *vanishing* o *exploding gradient*.

Concetto chiave: gradient clipping

Anche con buona inizializzazione, durante il training i gradienti possono "esplodere" a valori giganteschi a causa di catene di moltiplicazioni. Il *gradient clipping* è una protezione: se la norma del gradiente supera una soglia, si scala il gradiente per riportarlo entro la soglia. È implementato nella backpropagation del progetto.

Punti chiave da ricordare

- Una rete neurale è una composizione di trasformazioni lineari $\boldsymbol{W}\boldsymbol{x}+\boldsymbol{b}$ alternate a non linearità f.
- Le non linearità sono *necessarie*: senza, la rete è equivalente a un singolo layer lineare.
- Il forward step calcola la predizione; la loss misura l'errore; il backward step calcola il gradiente per scenderlo.
- Il learning rate η è l'iperparametro più critico della backpropagation: controlla la dimensione del passo.
- Backpropagation = regola della catena applicata in modo efficiente partendo dall'output.

4 Algoritmi Genetici

4.1 La metafora biologica

Gli **algoritmi genetici** (GA) sono una classe di algoritmi evolutivi ispirati alla selezione naturale di Darwin. L'idea fondante è che soluzioni a un problema possono "evolvere" nel tempo se sottoposte ai meccanismi della selezione naturale: i "più adatti" sopravvivono e si riproducono, i "meno adatti" tendono a scomparire.

Nel quadro di Russell & Norvig, un GA può essere visto come una variante della *ricerca locale stocastica*: si mantiene non un singolo stato candidato ma un'*intera popolazione* di stati, e si esplora lo spazio combinando coppie di stati invece di muoversi a piccoli passi da uno stato singolo.

Concetto chiave: un GA è caratterizzato da queste scelte

- Dimensione della *popolazione*.
- *Rappresentazione* di ogni individuo (stringa, lista, vettore...).
- Numero ρ di genitori per ogni figlio (tipicamente $\rho = 2$).
- Procedura di *selezione* dei genitori (a torneo, a roulette, ...).
- Procedura di *ricombinazione* (crossover).
- *Tasso di mutazione*.
- Strategia di formazione della nuova generazione (con o senza elitismo).

4.2 Componenti fondamentali

Individuo (cromosoma) Una soluzione candidata al problema. Nel progetto è una `lista di tuple`, dove ogni tupla `(n_neuroni, funzione)` rappresenta un hidden layer.

Gene Elemento atomico del cromosoma. Nel progetto, una singola tupla `(n_neuroni, funzione)`.

Popolazione Insieme di tutti gli individui in una generazione.

Generazione Una iterazione completa del ciclo evolutivo.

Fitness Funzione che misura la qualità di un individuo. È il "segnale evolutivo" che guida la ricerca: più alta è meglio.

4.3 Il ciclo evolutivo

Lo schema generale di un GA è il seguente:

Algoritmo 2 Algoritmo Genetico (schema generale)

```
Genera una popolazione iniziale casuale di N individui
repeat
    Valuta la fitness di ogni individuo
    Elitismo: copia il migliore nella nuova popolazione
    while nuova popolazione < N do
        Selezione: scegli due genitori dalla popolazione corrente
        Crossover: combina i due genitori per creare un figlio
        Mutazione: con probabilità p_m modifica il figlio
        Aggiungi il figlio alla nuova popolazione
    end while
    popolazione ← nuova popolazione
until condizione di terminazione (numero di generazioni o fitness target)
return l'individuo con fitness più alta mai trovato
```

4.4 Selezione: torneo

Esistono diverse procedure di selezione, ognuna con un diverso modo di bilanciare due esigenze opposte:

- **Sfruttamento (*exploitation*)** — i migliori individui devono avere più probabilità di riprodursi.
- **Esplorazione (*exploration*)** — anche individui mediocri devono avere qualche chance, per mantenere diversità genetica e ridurre il rischio di *convergenza prematura* su un ottimo locale.

Le tre procedure più comuni:

- **Roulette wheel** — ogni individuo ha probabilità di essere scelto proporzionale alla sua fitness. Semplice ma sensibile alla scala della fitness (un individuo molto migliore degli altri domina subito).
- **Selezione a torneo** — (usata nel progetto). Si estraggono k individui a caso dalla popolazione e si sceglie il migliore tra loro come genitore. Non sensibile alla scala assoluta della fitness, solo all'*ordinamento*: un individuo con fitness 0.5 ha la stessa probabilità di vincere su un avversario con fitness 0.4 che su uno con fitness 0.1.
- **Selezione per rango** — gli individui sono ordinati per fitness, le probabilità sono assegnate in base al rango (non al valore). Anche questa è insensibile alla scala.

Domanda tipo orale

"Perché selezione a torneo invece di scegliere sempre i migliori?"
Se selezionassi sempre i migliori, dopo poche generazioni l'intera popolazione diventerebbe quasi identica e il GA smetterebbe di esplorare, restando incastrato in un ottimo locale. Con il torneo $k = 3$, un individuo forte ha alta probabilità di vincere ma uno mediocre ha ancora chance se gli capita un torneo con avversari più deboli. La taglia k controlla il

bilanciamento: k grande $\Rightarrow$ più pressione selettiva, k piccolo $\Rightarrow$ più esplorazione.

4.5 Crossover

Il **crossover** (o *ricombinazione*) combina due genitori per generare un figlio. L'idea: se entrambi i genitori sono buoni, è probabile che combinazioni di parti di ciascuno producano figli ancora migliori.

La forma classica per stringhe di lunghezza fissa è il *single-point crossover*: si sceglie un punto di taglio casuale c, il figlio prende le posizioni $1 \ldots c$ dal genitore 1 e $c+1 \ldots n$ dal genitore 2.

```
genitore_1: [g1 g1 g1 | g1 g1 g1]   <- punto di taglio c
genitore_2: [g2 g2 g2 | g2 g2 g2]
figlio_1:   [g1 g1 g1 | g2 g2 g2]
```

Nel progetto i cromosomi hanno *lunghezza variabile* (architetture con numero di layer diverso), quindi serve una variante: si scelgono due punti di taglio indipendenti, t_1 per il primo genitore e t_2 per il secondo, e si concatena.

4.6 Mutazione

La **mutazione** introduce variazioni casuali nel cromosoma. È fondamentale per due motivi:

- Mantiene *diversità genetica* nella popolazione, evitando la convergenza prematura.
- Permette di esplorare zone dello spazio delle soluzioni che il crossover da solo non potrebbe raggiungere (il crossover ricombina ma non crea "materiale genetico" davvero nuovo).

Il *mutation rate* p_m è la probabilità che ciascun gene venga modificato. Valori tipici: 0.05–0.30.

4.6.1 I tre tipi di mutazione del progetto

Ogni gene è una tupla `(n_neuroni, funzione)`. Quando un gene viene mutato, una delle tre seguenti operazioni viene scelta con uguale probabilità:

1. **Mutazione del numero di neuroni** — si sostituisce `n_neuroni` con un nuovo valore casuale entro il range ammesso, mantenendo la stessa funzione.
2. **Mutazione della funzione** — si sostituisce la funzione di attivazione con un'altra del pool, mantenendo lo stesso numero di neuroni.
3. **Mutazione strutturale** — si aggiunge un nuovo layer in fondo o si rimuove uno dei layer esistenti (a meno che il cromosoma non abbia un solo layer, nel qual caso non si rimuove).

I primi due tipi raffinano un'architettura esistente; il terzo permette di esplorare *topologie* diverse (numero di layer variabile).

4.7 Elitismo

L'**elitismo** è una tecnica semplice e potente: il miglior individuo della generazione corrente viene copiato *integralmente* nella generazione successiva, senza subire crossover o mutazione. Questo

garantisce che la *best fitness* della popolazione non possa mai peggiorare da una generazione all'altra.

Concetto chiave: perché elitismo

Senza elitismo, il miglior individuo trovato finora potrebbe essere "perso" a causa della casualità di crossover/mutazione. Con elitismo, la ricerca è *monotona*: una volta trovata un'architettura buona, non viene più persa. Lo svantaggio è una leggera riduzione della diversità, ma in pratica copiare 1 individuo su 20 ha effetto trascurabile.

4.8 GA come variante della ricerca locale

Vale la pena collocare il GA nel quadro più ampio degli algoritmi di ricerca locale visto a lezione (slide "Ricerca in ambienti complessi"). Gli algoritmi di ricerca locale lavorano nel *problem space* senza mantenere il percorso, e cercano una soluzione migliorando un singolo stato (o un piccolo insieme di stati) tramite mosse locali.

- **Hill climbing** — a ogni passo va al successore con valore migliore. Veloce ma rischia ottimi locali.
- **Simulated annealing** — accetta anche mosse peggiorative con probabilità decrescente nel tempo. Permette di uscire da ottimi locali.
- **Local beam search** — mantiene k stati in parallelo, a ogni passo li espande tutti e tiene i migliori k successori.
- **Algoritmi genetici** — come il beam search ma con ricombinazione fra stati: i figli sono generati *combinando* due stati genitore, non da successori di un singolo stato.

Domanda tipo orale

"Cosa distingue un GA da un beam search?"
Il beam search mantiene k stati e a ogni passo genera i successori di ciascuno indipendentemente. Il GA aggiunge il *crossover*: i nuovi candidati possono combinare informazioni da più stati genitori. Questo rende il GA capace di mescolare "ingredienti buoni" provenienti da soluzioni diverse, cosa che il beam search non fa.

Punti chiave da ricordare

- Un GA è un algoritmo di ricerca evolutiva che lavora su una popolazione di soluzioni candidate.
- Le componenti fondamentali sono: rappresentazione, fitness, selezione, crossover, mutazione.
- Le scelte di progettazione del GA bilanciano *esplorazione* (mutazione, popolazione grande) e *sfruttamento* (selezione, elitismo).
- Il GA si colloca nella famiglia delle ricerche locali stocastiche, distinto da hill climbing e simulated annealing per la presenza della popolazione e del crossover.

5 Implementazione del progetto

5.1 Architettura del sistema

Il sistema è composto da due moduli principali che collaborano:

- Il modulo `neural_network` contiene la classe `neural_network` (motore feedforward + backpropagation) e le funzioni di attivazione.
- Il modulo `genetic_algorithm` contiene la classe `GeneticAlgorithm` che orchestra il ciclo evolutivo e chiama la rete come "oracolo" per valutare le architetture candidate.

Il flusso di alto livello è:

```
+---------------------------------------------+
| Carica dataset -> train / val / test split  |
+---------------------------+-----------------+
                            |
                            v
+---------------------------------------------+
| Inizializza popolazione casuale di N reti   |
+---------------------------+-----------------+
                            |
                            v
+---------------------------------------------+
| Per ogni generazione:                       |
|   1. valuta fitness di ogni individuo       |
|      (per ogni individuo: training BP +     |
|       accuracy su val, ripetuto K volte)    |
|   2. salva il migliore (elitismo)           |
|   3. genera nuova popolazione tramite       |
|      selezione + crossover + mutazione      |
+---------------------------+-----------------+
                            |
                            v
+---------------------------------------------+
| Riaddestra il migliore e valuta sul TEST    |
| set (stima onesta delle prestazioni)        |
+---------------------------------------------+
```

5.2 Rappresentazione del cromosoma

Un individuo è una lista Python di tuple, una per ogni hidden layer:

```
[(12, relu), (6, sigmoid)]              # 2 hidden layer
[(32, relu), (16, relu), (8, relu)]     # 3 hidden layer
[(8, linear)]                           # 1 hidden layer
```

- Il *numero* di tuple = numero di hidden layer (può variare da individuo a individuo).

- Per ogni tupla: `n_neuroni` (intero in un range fissato, es. 4–64) e `funzione` (scelta fra ReLU, Leaky ReLU, Sigmoid, Linear).
- Il layer di input e quello di output sono *determinati dal dataset* (numero di feature in input, numero di classi in output) e non fanno parte del cromosoma.
- La funzione di output è fissa a `softmax`, perché tutti i dataset usati sono di classificazione multiclasse.

Domanda tipo orale

"I pesi della rete fanno parte del cromosoma?"
No. Il cromosoma rappresenta solo l'*architettura* (struttura discreta). I pesi sono inizializzati casualmente *ogni volta* che si costruisce una rete per valutarla. La metafora biologica è precisa: i geni trasmettono le istruzioni per costruire l'organismo, non l'organismo già formato. L'architettura è il gene, i pesi sono le caratteristiche fisiche sviluppate durante il training.

5.3 Funzione di fitness

La fitness misura quanto è "buona" un'architettura. Questa è la formulazione finale usata nel progetto:

$$\text{fitness} = \text{accuracy} - \lambda \cdot \log_{10}(N_{\text{params}}) \tag{5}$$

dove accuracy è la percentuale di predizioni corrette sul validation set (mediata su K run indipendenti), N_{params} è il numero totale di pesi e bias dell'architettura, e $\lambda \geq 0$ è il coefficiente di penalità di complessità.

5.3.1 Perché penalizzare la complessità

Senza penalità ($\lambda = 0$) il GA tende a far crescere le architetture generazione dopo generazione, perché reti più grandi tendono ad avere accuracy leggermente superiore (maggiore capacità di adattarsi ai dati). Questo causa due problemi:

- Reti grandi sono più soggette a *overfitting* sul validation set (gap fra accuracy val e accuracy test maggiore).
- Reti grandi sono inutilmente costose da addestrare e usare.

La penalità λ è una forma di *regolarizzazione*: scoraggia la complessità per scelta progettuale.

5.3.2 Perché logaritmica e non lineare

Una prima versione del progetto usava penalità lineare, $\lambda \cdot N_{\text{params}}$. Funziona, ma il valore di λ utile dipende fortemente dalla scala delle reti generate: una stessa λ non penalizza praticamente nulla una rete piccola e annienta una rete grande.

Con penalità logaritmica, il valore della penalità cresce in modo molto più lento: passare da 100 a 10 000 parametri (un fattore $\times 100$) fa salire il termine di soli 2 punti ($\log_{10}(100) = 2$,

$\log_{10}(10000) = 4$). Questo rende λ stabile e indipendente dalla scala assoluta dei parametri.

5.3.3 Perché K valutazioni

I pesi della rete sono inizializzati casualmente a ogni valutazione, e con SGD a campioni casuali il training è di per sé stocastico. Conseguenza: una stessa architettura, addestrata due volte di seguito, può ottenere accuracy diverse anche di alcuni punti percentuali. Questo *rumore di valutazione* può rendere il segnale di fitness inaffidabile e portare il GA a selezionare individui "fortunati" piuttosto che individui realmente buoni.

La soluzione è ripetere la valutazione K volte e mediare:

$$\text{accuracy} = \frac{1}{K} \sum_{k=1}^{K} \text{accuracy}_k$$

Con $K = 3$ la deviazione standard si dimezza rispetto a $K = 1$ (vedi sezione esperimenti). Aumentare K oltre 3 riduce ulteriormente la varianza ma triplica o quintuplica il costo computazionale, con beneficio marginale.

Algoritmo 3 Valutazione della fitness di un individuo

Input: cromosoma c (architettura), training set, validation set
Output: (fitness, accuracy)

```
sum_acc ← 0
for k = 1 to K do
    Costruisci la rete N con architettura c e pesi inizializzati a caso
    for epoch = 1 to E do
        Estrai un campione casuale (x, y) dal training set
        Esegui un passo di backpropagation su (x, y)
    end for
    correct ← numero di predizioni corrette di N sul validation set
    acc_k ← correct / |validation set|
    sum_acc ← sum_acc + acc_k
end for
accuracy ← sum_acc/K
fitness ← accuracy − λ · log_10(#params(c))
return (fitness, accuracy)
```

Nota: fitness e accuracy vengono restituite *separatamente*. Il GA usa la fitness per la selezione, ma nei grafici si mostra l'accuracy pura per leggibilità.

5.4 Operatori genetici (pseudocodice)

5.4.1 Selezione a torneo

Algoritmo 4 `selezione_torneo`(popolazione, fitness, k)

1: Estrai casualmente k indici dalla popolazione
2: Ordina i k individui per fitness decrescente
3: **return** l'individuo con fitness più alta tra i k

Viene chiamata *due volte* per ogni figlio da generare (una per ogni genitore). Il parametro k (qui chiamato `tournament_size`) controlla la pressione selettiva.

5.4.2 Crossover a doppio punto di taglio

Algoritmo 5 `crossover`(genitore_1, genitore_2)

1: $t_1 \leftarrow$ punto di taglio casuale in $[0, |\text{gen}_1|]$
2: $t_2 \leftarrow$ punto di taglio casuale in $[0, |\text{gen}_2|]$
3: figlio $\leftarrow \text{gen}_1[: t_1] + \text{gen}_2[t_2 :]$
4: **if** figlio è vuoto **then** ▷ caso degenere
5: figlio $\leftarrow$ [un gene casuale dal pool dei due genitori]
6: **end if**
7: **return** figlio

Perché due punti di taglio diversi e non uno solo? Perché i due genitori possono avere lunghezze diverse: con un singolo t si rischierebbero combinazioni nonsensiche o sempre della stessa lunghezza. Con t_1, t_2 indipendenti il figlio può avere lunghezza qualsiasi tra 0 e $|gen_1| + |gen_2|$, esplorando bene lo spazio delle topologie.

5.4.3 Mutazione

Algoritmo 6 `mutazione`(individuo, p_m)

1: nuovo $\leftarrow$ copia di individuo
2: **for** ogni gene (n, f) in nuovo **do**
3: **if** rand $< p_m$ **then**
4: scegli casualmente uno dei tre tipi:
TIPO 1 sostituisci n con un nuovo valore casuale (mantieni f)
TIPO 2 sostituisci f con un'altra funzione casuale (mantieni n)
TIPO 3 aggiungi un nuovo gene casuale o rimuovi questo gene (se $|\text{nuovo}| > 1$)
5: **end if**
6: **end for**
7: **return** nuovo

5.5 Il ciclo principale (run)

Algoritmo 7 `GeneticAlgorithm.run()`

```
P ← popolazione iniziale di N individui casuali
Inizializza storia_best_fit, storia_best_acc, storia_mean_acc
best_global ← None
for g = 1 to generazioni do
    Per ogni ind ∈ P, calcola (fit, acc) = fitness(ind)
    Aggiorna best_global se trovato un migliore
    Salva statistiche della generazione g nelle storie
    P_new ← [ best della generazione ]                    ▷ elitismo
    while |P_new| < N do
        p1 ← selezione_torneo(P, k)
        p2 ← selezione_torneo(P, k)
        f ← crossover(p1, p2)
        f ← mutazione(f, p_m)
        Aggiungi f a P_new
    end while
    P ← P_new
end for
return (best_global, storie)
```

Concetto chiave: parallelizzazione delle valutazioni

La valutazione della fitness di ogni individuo è la fase più costosa dell'algoritmo (richiede K training completi). Gli individui di una stessa generazione sono indipendenti fra loro, quindi possono essere valutati *in parallelo*. Nel progetto si usa `ProcessPoolExecutor` di Python per distribuire le valutazioni su più CPU.

5.6 Preprocessing del dataset

Tre operazioni di preprocessing standard, in questo ordine:

5.6.1 Split train / validation / test

Il dataset viene diviso in tre sottoinsiemi disgiunti, tipicamente con proporzioni 60%/20%/20%. Lo split è **stratificato**: la proporzione delle classi è preservata in ogni sottoinsieme. Il GA valuta la fitness sul *validation*; il *test* viene toccato solo alla fine, una volta sola, per riportare il risultato finale.

5.6.2 Normalizzazione (StandardScaler)

Le feature vengono normalizzate per avere media 0 e deviazione standard 1:

$$x' = \frac{x - \mu}{\sigma}$$

Questo aiuta la backpropagation perché tutte le feature operano sulla stessa scala (altrimenti pesi diversi devono compensare scale diverse).

Attenzione

μ e σ vanno calcolati *solo sul training set*. Si applicano poi gli stessi valori (senza ricalcolarli) al validation e al test set. Calcolarli sull'intero dataset è una forma di *data leakage*: informazione del test set si "infiltra" nel training, gonfiando le stime di performance. Questo è oggetto del Test 9B.

5.6.3 One-hot encoding

Le etichette di classe (interi $0, 1, 2, \ldots$) vengono convertite in vettori binari con un solo 1 nella posizione corrispondente alla classe:

$$y = 2 \;\rightarrow\; \boldsymbol{y} = [0,\ 0,\ 1,\ 0,\ \ldots,\ 0]$$

Questo è necessario perché la rete ha tanti neuroni di output quante sono le classi, e la softmax produce una distribuzione di probabilità su tutte le classi.

5.7 La baseline di confronto

Per dare senso ai numeri prodotti dal GA, ogni esperimento confronta il risultato con una **baseline**: una rete con architettura *fissa* addestrata con backpropagation classica, senza l'aiuto del GA. La baseline utilizzata è $[(8, \text{leaky_relu}), (8, \text{leaky_relu})]$.

- Se il GA non batte la baseline, l'intero costo computazionale della ricerca evolutiva non è giustificato.
- Se il GA la batte, il valore aggiunto della ricerca automatica dell'architettura è dimostrato concretamente.

Per equità, la baseline è addestrata per lo stesso numero di epoche totali usato per valutare un singolo individuo del GA: $K \cdot E$ epoche (es. $3 \cdot 500 = 1500$).

Il numero di parametri della baseline varia a seconda del dataset (per via del diverso numero di feature in input e di classi in output):

dataset	feat. in	classi out	parametri baseline
iris	4	3	139
wine	13	3	211
breast_cancer	30	2	338
digits	64	10	682

Nei test di sensibilità (Test 1–7) il dataset è sempre `digits`, quindi il confronto è sempre rispetto a **682 parametri**.

Punti chiave da ricordare

- Il sistema è composto dal modulo `neural_network` (motore) e `genetic_algorithm` (orchestratore).
- Il cromosoma è una lista di tuple `(n_neuroni, funzione)`, una per hidden layer.
- La fitness è accuracy $- \lambda \cdot \log_{10}(N_{\text{params}})$, mediata su K run.
- Crossover a doppio punto di taglio (per gestire lunghezze variabili), mutazione di 3 tipi (numero, funzione, struttura), elitismo.
- Lo split è stratificato e la normalizzazione *si calcola solo sul training set* per evitare data leakage.
- La baseline è una rete fissa che permette di misurare se il GA aggiunge valore.

6 Concetti chiave trasversali

Questa sezione raccoglie alcuni concetti che attraversano il progetto e che molto probabilmente verranno chiesti all'orale. Ognuno è collegato all'una o all'altra delle scelte progettuali.

6.1 Overfitting nel contesto del progetto

Il GA, lasciato senza vincoli, ha una naturale tendenza a far crescere le architetture. Più una rete è grande, più parametri liberi ha, e più è capace di adattarsi (anche *troppo*) ai dati. Sul training set questo si traduce in accuracy più alta; sul validation set anche, ma in misura minore; sul test set spesso *peggio*.

Attenzione

Troppi parametri ⇒ maggiore rischio di overfitting.
Utilizzare una rete con molti più parametri di quanti ne servano realmente *aumenta* la probabilità di overfitting. Ecco perché:

- Ogni parametro è un grado di libertà in più che il modello può usare per adattarsi ai dati.
- Se i parametri sono più del necessario, quelli "in eccesso" non hanno pattern reali da imparare: finiscono per memorizzare il rumore specifico del training set.
- Questo si traduce in alta accuracy sul training ma generalizzazione peggiore sul test — la definizione classica di overfitting.

Esempio concreto (Test 8 — Wine): la baseline usa 211 parametri, il GA solo 122, entrambi raggiungono 100%. I 89 parametri in eccesso della baseline non portano nessun beneficio di accuracy, ma su un dataset più rumoroso o con meno campioni sarebbero liberi di memorizzare pattern spurii. La rete GA, più compatta, avrebbe un rischio di overfitting strutturalmente inferiore.

Concetto chiave: overfitting sul validation set

È una forma sottile di overfitting: il GA non sta facendo overfitting sul *training* (la backpropagation lo gestisce), sta facendo overfitting sul *validation set* perché lo usa come "oracolo" per scegliere l'architettura. Una rete che ha avuto fortuna in qualche peculiarità del validation viene selezionata, ma quella fortuna non si trasferisce al test set.

La penalità di complessità λ è la difesa diretta: scoraggia il GA dal selezionare reti grandi che hanno qualche frazione di punto di accuracy in più ma molti parametri in più. Il Test 4 (sezione successiva) mostra empiricamente questo effetto: senza penalità il GA seleziona reti con migliaia di parametri; con $\lambda = 0.05$ seleziona la stessa architettura ottima ma più snella.

6.2 Data leakage

Il **data leakage** è qualunque situazione in cui informazione del validation o test set si "infiltra" nel processo di training, gonfiando artificialmente le stime di performance. Le forme più comuni:

- **Leakage da preprocessing** — calcolare μ, σ (StandardScaler) o altre statistiche sull'*intero* dataset invece che solo sul training set. Le statistiche del test "contaminano" la trasforma-

zione applicata al training. È oggetto del Test 9B.

- **Leakage temporale** — in serie temporali, usare dati futuri per predire il passato. Non rilevante per Iris/Digits ma critico in finanza.
- **Bias ottimistico val vs test** — selezionare un modello in base alla sua performance sul validation set e poi riportare quella stessa performance come "risultato". Il modello scelto è per costruzione il più fortunato sul validation, quindi la stima è ottimistica. Oggetto del Test 9A.

Concetto chiave: la regola d'oro per evitare data leakage

Tutto ciò che richiede statistiche dei dati (medie, varianze, trasformazioni) si calcola *esclusivamente* sul training set, e si applica al validation/test *senza ricalcolare nulla*. Il test set si tocca *una sola volta*, alla fine.

6.3 Esplorazione vs sfruttamento

Quasi ogni iperparametro del GA si può leggere come una scelta sul trade-off tra esplorazione e sfruttamento:

Iperparametro	Verso esplorazione	Verso sfruttamento
population_size	grande (più diversità)	piccolo (concentrazione)
mutation_rate	alto (più variazioni)	basso (preserva i buoni)
tournament_size	piccolo (più chance ai meno forti)	grande (i migliori dominano)
elitismo	senza (più churn)	con (preserva il top)

Domanda tipo orale

"Cosa succede se il mutation rate è troppo alto?"
Il GA si riduce a una ricerca quasi casuale. Ogni generazione modifica così tanto i figli che le caratteristiche buone trovate dai genitori vanno perse. Empiricamente, con $p_m \geq 0.4$ la *mean accuracy* della popolazione fatica a salire e diventa rumorosa: il segnale evolutivo si perde.

6.4 Generalizzabilità: metodo vs risultato

Domanda tipo orale

"L'architettura trovata su Digits funzionerebbe su altri dataset?"
No. Il GA trova la migliore architettura *per quel dataset specifico*. L'architettura ottimale dipende dalla complessità del problema (numero di feature, separabilità delle classi, quantità di dati). Il *metodo* (GA + NAS) è generalizzabile, ma il *risultato* (l'architettura trovata) è specifico per il dataset. Su problemi più complessi servirebbero più generazioni, popolazione più grande e architetture più profonde. Il Test 8 misura proprio questo, eseguendo lo stesso GA su quattro dataset diversi e confrontando le architetture trovate.

6.5 Ottimizzazione annidata: due livelli

Concetto chiave: GA + backpropagation = due ottimizzatori in cascata

Nel progetto convivono due algoritmi di ottimizzazione che operano su scale e parametri diversi:

- **Livello esterno** (lento, su parametri *discreti*): il GA ottimizza l'*architettura* (numero di layer, neuroni, funzioni di attivazione). Lavora su una popolazione, una generazione alla volta.
- **Livello interno** (veloce, su parametri *continui*): la backpropagation ottimizza i *pesi* di ogni rete candidata. Lavora con SGD su un singolo individuo, una epoca alla volta.

La fitness del GA include la performance ottenuta dopo che il livello interno è terminato. È un caso classico di *bilevel optimization*.

Domanda tipo orale

"Perché non far evolvere anche i pesi al GA?"
Perché lo spazio dei pesi è continuo e ad altissima dimensionalità (decine di migliaia di parametri reali per una rete media). Il GA è efficiente su spazi discreti e di dimensione moderata, ma è enormemente inefficiente sui pesi: la backpropagation, che usa l'informazione del gradiente, è ordini di grandezza più rapida. La filosofia "ognuno fa quello che sa fare meglio": GA sui parametri discreti (struttura), backpropagation sui parametri continui (pesi).

Punti chiave da ricordare

- Il GA fa overfitting sul *validation set*, non sul training: la penalità λ lo previene.
- Tre forme di data leakage da conoscere: preprocessing, temporale, bias val/test.
- Quasi ogni iperparametro del GA è un controllo del trade-off esplorazione/sfruttamento.
- Il metodo (GA + NAS) è generale, ma l'architettura trovata è specifica del dataset.
- Il progetto è un caso di *ottimizzazione annidata*: GA sui parametri discreti, backpropagation sui pesi continui.

7 Esperimenti e Risultati

7.1 Metodologia: analisi di sensibilità

L'obiettivo degli esperimenti è rispondere a una domanda: *quali valori degli iperparametri del GA producono il miglior compromesso tra qualità della soluzione, complessità della rete e costo computazionale?*

La metodologia adottata è l'**analisi di sensibilità**: si varia *un iperparametro alla volta* tenendo fissi tutti gli altri, osservando l'effetto netto sulla `test_accuracy` e su `n_params`. I test vengono eseguiti in un ordine preciso (ottimizzazione sequenziale): ogni volta che un parametro viene fissato, i test successivi usano quel valore aggiornato.

7.1.1 Dataset principale: `load_digits`

Tutti i test di sensibilità sono condotti sul dataset `load_digits` di scikit-learn:

- 1797 immagini 8×8 di cifre scritte a mano (riconoscibili come $0 \ldots 9$);
- 64 feature continue (intensità dei pixel);
- 10 classi (le cifre);
- complessità: alta. È stato scelto come dataset principale *proprio perché* è abbastanza complesso da rendere significative le differenze fra architetture (su Iris, troppo semplice, ogni rete ragionevole arriva al 90%+).

Solo nel Test 8 si confrontano i risultati su quattro dataset diversi (Iris, Wine, Breast Cancer, Digits) per testare la generalizzabilità del metodo.

7.1.2 Configurazione di base

I valori di partenza sono volutamente conservativi (popolazione e generazioni piccole per contenere il tempo di esecuzione):

Parametro	Valore di partenza
population_size	20
generations	20
mutation_rate	0.2
tournament_size	5
epochs	150
learning_rate	0.01
lambda_	0.05
K	5
seed	42

L'ordine di ottimizzazione: `epochs` → `K` → `learning_rate` → `lambda` → `mutation_rate` → `population` → `generations`, seguito dai test 8 (generalizzabilità) e 9 (data leakage).

7.2 Test 1 — Numero di epoche

Ipotesi. Poche epoche ⇒ la rete non converge ⇒ fitness rumorosa ⇒ il GA seleziona architetture "per caso". Molte epoche ⇒ stima accurata ma costosa.

epochs	test_accuracy	tempo (s)	n_params GA
100	53.72%	96.6	4 735
250	78.33%	96.8	4 435
500	88.06%	100.3	3 910
1000	91.44%	129.3	4 360

Baseline Digits: ***682 parametri.***

Osservazioni. Con 100 epoche la `test_accuracy` è 53.72% — appena meglio di un classificatore casuale su 10 classi (10%). La rete non ha avuto tempo di convergere e il segnale di fitness è completamente inutilizzabile. A 500 epoche si ottiene 88.06% con un overhead di soli 4 secondi rispetto a 250: il salto è enorme. Da 500 a 1000 epoche il guadagno è di +3.4 punti percentuali ma il tempo cresce del 29% (rendimenti decrescenti).

Con poche epoche il GA seleziona architetture con 4 000+ parametri, molto più grandi della baseline (682): il segnale di fitness è rumoroso e il GA non riesce a discriminare architetture compatte da quelle grandi, quindi le architetture crescono liberamente. Solo con epoche sufficienti la fitness diventa affidabile e la penalità λ può fare effetto, portando a reti più snelle.

Scelta finale: `epochs = 500`. Punto di massimo rendimento marginale.

7.3 Test 2 — Numero di valutazioni K

Ipotesi. La fitness è la media di K run indipendenti con seed diversi. K basso ⇒ stima rumorosa. K alto ⇒ stima stabile ma ogni generazione costa K volte di più.

K	mean_test	std_test	n_params GA
1	85.61%	1.78%	4 135
3	85.70%	0.93%	3 760
5	86.45%	1.64%	3 910
10	86.86%	0.79%	3 910
20	86.78%	0.93%	4 735

Baseline Digits: ***682 parametri.***

Osservazioni. Con $K = 1$ la deviazione standard sulla `test_accuracy` (calcolata su 5 seed diversi) è 1.78%: una rete può ricevere una valutazione falsamente alta o bassa per pura casualità del seed. Con $K = 3$ la deviazione si dimezza a 0.93% a costo solo triplicato. Da $K = 3$ a $K = 10$ la `std` scende a 0.79%, miglioramento marginale di 0.14 punti che non giustifica il triplicarsi del costo.

In questo test il GA trova architetture con 3 700–4 700 parametri, molto più grandi della baseline (682): il learning rate è ancora a 0.01 (subottimale) e la fitness rumorosa impedisce alla penalità λ di operare efficacemente, lasciando crescere le architetture.

Scelta finale: $K = 3$. Dimezza il rumore di valutazione a costo computazionale ragionevole.

7.4 Test 3 — Learning rate

Ipotesi. LR troppo basso ⇒ la rete non converge in 500 epoche; LR troppo alto ⇒ oscillazioni o instabilità numerica.

learning_rate	test_accuracy	architettura trovata	n_params GA
0.001	38.15%	[(63, relu)]	4 735
0.005	81.48%	[(64, relu)]	4 810
0.01	87.31%	[(50, leaky_relu)]	3 760
0.05	90.65%	[(24, relu)]	1 810
0.1	90.74%	[(24, sigmoid)]	1 810

Baseline Digits: ***682 parametri.***

Osservazioni.

- Con $\eta = 0.001$ la rete non converge in 500 epoche (38.15% è poco più di un classificatore casuale): i passi di gradient descent sono troppo piccoli.
- Con $\eta = 0.05$ e $\eta = 0.1$ si ottiene accuracy quasi identica, ma con $\eta = 0.1$ il GA seleziona un'architettura con `sigmoid`: questa funzione satura facilmente quando la LR è alta, introducendo instabilità nei test successivi.
- Con $\eta = 0.05$ il GA trova `[(24, relu)]` in modo stabile e riproducibile.
- **Il learning rate influenza i parametri trovati:** con LR basso (≤ 0.01) il GA seleziona architetture con 3 700– 4 800 parametri (fino a 7× la baseline). Con LR ottimale (0.05) converge a 1 810 parametri. La ragione è che con LR basso il segnale di fitness è rumoroso e il GA non riesce a discriminare reti compatte da quelle grandi — le architetture crescono senza controllo.

Concetto chiave: il learning rate è il parametro più critico

Su tutti i parametri testati, il learning rate produce la differenza più ampia tra caso peggiore e caso migliore (oltre 50 punti percentuali!). È il parametro su cui si gioca tutto: una scelta sbagliata rende il GA completamente inutile, perché il segnale di fitness diventa rumore puro.

Scelta finale: `learning_rate = 0.05`. Massima accuracy stabile con architettura più snella.

7.5 Test 4 — Penalità di complessità λ

Ipotesi. $\lambda = 0$ ⇒ il GA seleziona reti grandi (overfitting sul validation, gap val/test elevato). λ alto ⇒ reti minuscole con accuracy insufficiente.

λ	test_accuracy	n_params	architettura
0.0	90.65%	1 810	[(24, relu)]
0.01	90.65%	1 810	[(24, relu)]
0.05	90.65%	1 810	[(24, relu)]
0.1	88.80%	1 135	[(15, leaky_relu)]
0.2	86.20%	760	[(10, relu)]

Baseline Digits: ***682 parametri.***

Osservazioni.

- Da $\lambda = 0$ a $\lambda = 0.05$ la `test_accuracy` è identica e la stessa architettura viene selezionata (1 810 parametri). La penalità c'è ma non è abbastanza forte da distorcere la scelta: 1 810 parametri sono più che necessari per Digits, ma la penalità logaritmica non li penalizza abbastanza da spingersi verso reti più piccole.
- Da $\lambda = 0.1$ in poi la `test_accuracy` cala (-1.85%, poi -4.45%) perché il GA preferisce reti troppo piccole: 1 135 e 760 parametri sono insufficienti per la complessità di Digits.
- La baseline con 682 parametri ottiene solo 58.33%, a conferma che su Digits servono più parametri di quelli che la baseline fissa può offrire.

Scelta finale: $\lambda = 0.05$. Mantiene tutta l'accuracy disponibile e introduce un incentivo alla semplicità che agisce sui casi borderline (es. quando due architetture hanno accuracy molto simile, quella con meno parametri viene preferita).

Domanda tipo orale

"Perché penalità logaritmica e non lineare?"
Con la penalità lineare $\lambda \cdot N_{\text{params}}$ il valore ottimale di λ dipende dalla scala assoluta delle reti generate. Una stessa λ su una rete da 100 parametri quasi non fa nulla, sulla stessa rete a 10000 parametri annienta tutto. Con $\log_{10}$ il fattore di crescita della penalità è mite (passare da 100 a 10000 parametri aumenta la penalità solo di 2), rendendo λ stabile indipendentemente dalla scala.

7.6 Test 5 — Mutation rate

Ipotesi. Rate basso $\Rightarrow$ il GA si appoggia solo al crossover, rischia ottimi locali. Rate alto $\Rightarrow$ il GA si riduce a ricerca casuale.

p_m	test_accuracy	n_params	architettura
0.0	89.07%	2 110	[(28, leaky_relu)]
0.05	89.07%	2 110	[(28, leaky_relu)]
0.1	90.19%	1 885	[(25, relu)]
0.2	90.65%	1 810	[(24, relu)]
0.4	90.19%	1 885	[(25, relu)]
0.8	91.11%	1 735	[(23, relu)]

Baseline Digits: ***682 parametri.***

Osservazioni.

- Con $p_m = 0$ e $p_m = 0.05$ il GA converge sulla stessa architettura (28 neuroni, leaky_relu): senza mutazione significativa, il crossover da solo rimane intrappolato in un ottimo locale.
- $p_m = 0.2$ è il picco tra i valori *stabili*.
- $p_m = 0.8$ produce una `test_accuracy` apparentemente migliore, ma con l'80% dei geni mutati ad ogni passo il GA non costruisce più sull'evoluzione precedente: i risultati sono altamente dipendenti dal seed e poco riproducibili.

Scelta finale: $p_m = 0.2$. Il rate più conservativo che raggiunge il picco di `test_accuracy` con comportamento stabile.

7.7 Test 6 — Dimensione della popolazione

Ipotesi. Popolazione piccola ⇒ ricerca ristretta, convergenza prematura. Popolazione grande ⇒ esplorazione ampia ma con generazioni fissate ogni individuo viene raffinato meno.

population	test_accuracy	gap val−test	n_params
5	87.69%	3.33%	1 585
10	88.98%	1.85%	1 510
20	90.65%	0.83%	1 810
30	89.07%	2.41%	2 110
50	89.63%	1.30%	1 360
70	90.00%	1.67%	1 960
100	88.80%	0.94%	1 135

Baseline Digits: ***682 parametri.***

Osservazioni. La relazione *non* è monotona: c'è un picco netto a popolazione = 20, sia in `test_accuracy` che in gap val/test minimo. Il gap val/test minimo a 0.83% è una doppia conferma — non è un artefatto della validation set.

Con popolazioni grandi e numero di generazioni fisso, ogni generazione esplora più candidati ma li raffina meno (la pressione selettiva è diluita). Con popolazioni piccole, manca diversità per uscire da ottimi locali. Il punto di equilibrio è 20.

Scelta finale: `population = 20`.

7.8 Test 7 — Numero di generazioni

Ipotesi. Poche generazioni ⇒ il GA non ha avuto tempo di convergere. Troppe ⇒ costo computazionale inutile perché la fitness è già stabile.

generations	test_accuracy	architettura	n_params GA
5	88.61%	[(52, relu)]	3 910
10	90.65%	[(24, relu)]	1 810
20	90.65%	[(24, relu)]	1 810
30	90.65%	[(24, relu)]	1 810
50	90.65%	[(24, relu)]	1 810
70	90.65%	[(24, relu)]	1 810
100	90.65%	[(24, relu)]	1 810

Baseline Digits: ***682 parametri****.*

Osservazioni. La convergenza è sorprendentemente rapida: già a gen = 10 il GA trova la stessa architettura ottimale che trova anche a gen = 100. Il salto da 5 a 10 è l'unico significativo (+2.04%); da 10 in poi il risultato è identico. Da gen = 10 in poi anche il numero di parametri si stabilizza a 1 810: l'architettura è convergita.

Scelta finale: `generations = 50`**.** Ampio margine di sicurezza sul punto di convergenza reale (gen = 10), senza sprecare tempo oltre.

7.9 Test 8 — Generalizzabilità su dataset diversi

Ipotesi. La configurazione ottimale trovata su Digits non è sovra-specializzata per quel dataset, ma funziona ragionevolmente anche su altri.

dataset	feat.	cl.	GA test%	Baseline%	vince (acc.)?
iris	4	3	92.22%	94.44%	no
wine	13	3	100.00%	100.00%	pari*
breast_cancer	30	2	95.61%	96.20%	no
digits	64	10	90.65%	58.33%	sì (+32%)

* *Accuracy identica ma architetture molto diverse per numero di parametri: vedere tabella sotto.*

Confronto parametri: GA vs baseline.

dataset	params GA	architettura GA	params BL	rapporto
iris	43	[(5, leaky_relu)]	139	GA usa 31%
wine	122	[(7, relu)]	211	GA usa 58%
breast_cancer	167	[(5, relu)]	338	GA usa 49%
digits	1 810	[(24, relu)]	682	GA usa 266%

Osservazioni.

- Su *Iris* e *Breast Cancer* la baseline batte il GA di pochi decimi di punto in accuracy, ma usa 3–4.5× più parametri. Il GA trova soluzioni più parsimonious (43 vs 139 e 167 vs 338): a parità quasi completa di performance, la rete più piccola è da preferire perché generalizza meglio su dati rumorosi.
- Su *Wine* entrambi raggiungono il 100%, ma il GA ottiene lo stesso risultato con soli 122

parametri contro i 211 della baseline (−42%). La parità di accuracy *non significa* parità di soluzioni: la baseline ha 89 parametri in eccesso rispetto a quanti ne servono realmente. Questi parametri "in più" non portano benefici, e su dati più rumorosi o con meno campioni aumenterebbero il rischio di overfitting (vedi § 6.1).

- Su *Digits* il GA batte la baseline di 32 punti percentuali e giustamente usa 1 810 parametri (2.7× la baseline). La baseline con 682 parametri è *sottodimensionata* per la complessità del problema (64 feature, 10 classi): non ha abbastanza capacità e rimane a 58.33%. Il GA esplora lo spazio e trova la dimensione giusta.

Concetto chiave: la NAS aggiunge valore quando il problema è non triviale

Su dataset semplici, una rete fissa ragionevole è già adeguata: la NAS non guadagna molto in accuracy, ma trova comunque reti *più compatte*. Su dataset complessi, la ricerca automatica può fare una differenza enorme sia in accuracy che nel trovare la capacità giusta. Questo è anche il motivo per cui la NAS è particolarmente utile in applicazioni industriali (visione artificiale, NLP) dove i problemi sono complessi e la dimensione ottimale della rete non è ovvia.

Osservazione di rilievo. Su *tutti* i dataset il GA trova architetture a un solo hidden layer. Non è un limite del GA: è una risposta ai dati. Per questi problemi un singolo layer nascosto è sufficiente, e la penalità λ disincentiva la profondità inutile.

7.10 Test 9A — Bias ottimistico val vs test

Ipotesi. Il GA seleziona l'architettura migliore in base all'accuracy sul *validation* set. Riportare quella stessa accuracy come "risultato" del GA è ottimistico per costruzione.

metrica	valore (media su 10 seed)
`val_accuracy`	91.33%
`test_accuracy`	87.01%
gap medio	**4.33%**

Osservazioni. Il gap di 4.33% è sistematico. Su 10 seed diversi varia da 1.39% a 9.35%, ma è sempre positivo (la val è sempre più alta della test). Conclusione: la `test_accuracy` è la stima onesta da riportare; la `best_accuracy` (val) è ottimistica per costruzione.

Concetto chiave: lezione operativa

Quando si fa NAS o tuning di iperparametri tramite validation set, è *obbligatorio* avere un test set separato per la stima finale. Riportare la metrica usata per scegliere significa riportare il caso fortunato, non la performance attesa.

7.11 Test 9B — Leakage da preprocessing

Ipotesi. Calcolare le statistiche dello `StandardScaler` sull'intero dataset (train + val + test) anziché solo sul training introduce leakage e gonfia la stima.

scenario	test_accuracy
scaler corretto (solo train)	75.33%
scaler su tutto il dataset	74.00%

Osservazioni. In questo caso specifico, il leakage non ha gonfiato i risultati (anzi li ha leggermente peggiorati). Il motivo è che il dataset usato per questo test (Iris) è piccolo e omogeneo: la distribuzione dei dati è molto simile in ciascuno split, quindi le statistiche calcolate sull'intero dataset coincidono praticamente con quelle calcolate solo sul training.

Su dataset più grandi o con distribuzione eterogenea il leakage sarebbe più evidente: l'esperimento documenta il fatto che la *prassi* (scaler solo sul training) va comunque adottata sempre, anche quando nei casi piccoli non sembra fare differenza.

7.12 Riepilogo della configurazione ottimale

Parametro	Valore	Motivazione
epochs	500	Minimo per segnale fitness affidabile (rendimenti decrescenti oltre)
K	3	Dimezza la varianza di K=1 a costo minimo
learning_rate	0.05	Picco di test_accuracy con convergenza stabile (più critico di tutti)
lambda	0.05	Penalizza la complessità senza ridurre l'accuracy
mutation_rate	0.2	Picco stabile, evita ottimi locali senza diventare ricerca casuale
population	20	Picco di test_accuracy + minimo gap val/test
generations	50	Ampio margine sul punto di convergenza reale (gen=10)

Risultato finale su Digits: architettura `[(24, relu)]` — 1 hidden layer, 24 neuroni, ReLU.
Test accuracy: 90.65% vs baseline 58.33% (+32.32 punti percentuali).

Punti chiave da ricordare

- Il *learning rate* è di gran lunga il parametro più critico: una scelta sbagliata rende il GA inutile.
- Il numero di *epoche* controlla la qualità del segnale di fitness; troppo poche ⇒ rumore puro.
- K (numero di valutazioni) e λ (penalità) controllano rispettivamente rumore e complessità.
- *population* e *generations* contano meno di quanto ci si aspetterebbe: il GA converge presto.
- Il GA aggiunge valore reale solo su dataset *complessi*: su Iris la baseline è già adeguata.

- Test set indispensabile per stima onesta: la val accuracy è ottimistica di $\sim 4\%$ in media.

8 Discussione e Conclusioni

8.1 Cosa il progetto dimostra

Il progetto dimostra empiricamente, su un dataset non triviale, che **un Algoritmo Genetico è uno strumento efficace per la Neural Architecture Search**. Sul dataset Digits il GA scopre automaticamente un'architettura ($[(24, \text{relu})]$) che batte una baseline manuale ragionevole di 32 punti percentuali. La ricerca converge in poche generazioni e produce architetture compatte (un solo layer nascosto), un risultato non ovvio a priori.

Più in generale, il progetto mostra che:

- È possibile combinare due paradigmi diversi dell'AI (ricerca evolutiva ed apprendimento supervisionato) in un sistema coerente.
- L'*ottimizzazione annidata* (GA sopra, backpropagation sotto) funziona purché ognuno dei due livelli sia configurato correttamente.
- Il principio di *regolarizzazione* via penalità di complessità mantiene le architetture trovate snelle e robuste, riducendo l'overfitting sul validation set.

8.2 Limiti del progetto

- **Generalizzabilità del risultato.** Le architetture ottimali trovate sono *specifiche per il dataset* su cui sono state cercate. Il *metodo* è generale; il *risultato* no.
- **Spazio di ricerca limitato.** Il GA cerca solo nel sottospazio delle reti feed-forward MLP. Architetture convoluzionali, ricorrenti o con *skip connections* non sono nello spazio di ricerca. Questo è una limitazione di implementazione, non concettuale.
- **Iperparametri di addestramento esclusi.** Learning rate ed epochs sono iperparametri della backpropagation, fissati all'esterno del GA. Una versione più ambiziosa potrebbe includerli nel cromosoma.
- **Costo computazionale.** L'unica vera "commodità" del progetto. Su un singolo dataset con la configurazione finale, una run completa richiede dell'ordine dei minuti su CPU moderna. Per dataset molto grandi (ImageNet) approcci più sofisticati (NAS basato su reinforcement learning, weight sharing) sono necessari.
- **Variabilità tra run.** Anche con $K = 3$, esecuzioni con seed diversi possono trovare architetture leggermente diverse. Il sistema è stocastico per natura: non c'è un'unica "risposta giusta" a ogni esecuzione.

8.3 Sviluppi futuri

- **Estendere il cromosoma.** Includere learning rate, dimensione del batch, dropout rate come geni. Il GA ottimizzerebbe simultaneamente architettura e training.
- **Operatori di crossover più intelligenti.** Crossover consapevoli della semantica (es. scambiare layer simili in posizione simile) potrebbero accelerare la convergenza.

- **Funzione di fitness multi-obiettivo.** Invece di combinare accuracy e complessità in un'unica fitness scalare, esplorare il fronte di Pareto con un GA multi-obiettivo (NSGA-II).
- **Fitness stimata.** Il collo di bottiglia è il training completo per ogni valutazione. Un *fitness predictor* (a sua volta una rete neurale) potrebbe stimare la fitness senza training, accelerando l'evoluzione di ordini di grandezza.
- **Spazi di ricerca più ricchi.** Estendere il GA a gestire architetture non lineari (skip connection, branching).

9 Domande tipo orale (con risposte ragionate)

Questa sezione raccoglie le domande più probabili per l'orale, con risposte sintetiche ma complete. Le domande sono organizzate per argomento.

9.1 Sul progetto in generale

Domanda tipo orale

"Spiegami il tuo progetto in due minuti."
Ho implementato un sistema di Neural Architecture Search: un Algoritmo Genetico cerca automaticamente la migliore architettura di una rete neurale (numero di layer, neuroni, funzioni di attivazione) per un dato dataset. Ogni architettura candidata viene valutata addestrandola con backpropagation e misurando l'accuratezza sul validation set, penalizzata per la complessità della rete. Sul dataset Digits il sistema ha trovato un'architettura con un singolo layer da 24 neuroni ReLU, 90.65% di accuracy sul test set, contro il 58.33% di una baseline manuale. Sia la rete che il GA sono scritti da zero in numpy.

Domanda tipo orale

"Perché hai scelto un GA invece di altri metodi di NAS?"
Per tre motivi: (1) il GA è naturale per spazi di ricerca discreti come quello delle architetture; (2) gestisce nativamente cromosomi di lunghezza variabile (architetture con numero di layer diverso); (3) è algoritmicamente semplice da implementare da zero, coerente con l'obiettivo didattico del progetto. Alternative come grid search sono esponenzialmente costose, e reinforcement learning per NAS richiede infrastruttura molto più pesante.

Domanda tipo orale

"Quali sono le componenti principali del codice?"
Due moduli: `neural_network` contiene la classe della rete con forward, backward e una libreria di funzioni di attivazione; `genetic_algorithm` contiene la classe del GA con i metodi privati per fitness, selezione, crossover, mutazione, e il metodo pubblico `run()` che orchestra il ciclo evolutivo. C'è poi un `main.py` che fa preprocessing del dataset, esegue il GA e produce i grafici, e una serie di script di test che variano un parametro alla volta per l'analisi di sensibilità.

9.2 Su reti neurali

Domanda tipo orale

"Cos'è la backpropagation, in poche frasi?"
È l'algoritmo che calcola il gradiente della loss rispetto a tutti i pesi della rete, applicando la regola della catena layer per layer all'indietro a partire dall'output. Una volta ottenuto il gradiente, i pesi vengono aggiornati nella direzione opposta tramite gradient descent. Si chiama *back*-propagation perché il calcolo procede dall'*output verso l'input*, riusando i risultati intermedi salvati durante il forward pass per essere efficiente.

Domanda tipo orale

"A cosa serve la funzione di attivazione non lineare?"
A rendere la rete capace di rappresentare funzioni non lineari. Senza attivazioni non lineari, una rete con N layer è matematicamente equivalente a una rete con un singolo layer lineare: la composizione di trasformazioni lineari è lineare. È la non linearità a dare alla rete il suo potere rappresentativo (universal approximation theorem).

Domanda tipo orale

"Perché si usa softmax in output e non sigmoid?"
La softmax produce una distribuzione di probabilità sulle classi (i valori sommano a 1): è il modo naturale di esprimere "il modello crede che questo input appartenga alla classe i con probabilità p_i". Sigmoid produrrebbe valori indipendenti per ogni classe, non collegati fra loro. Per classificazione binaria si usa sigmoid; per multiclasse softmax.

Domanda tipo orale

"Cos'è il vanishing gradient e come si previene?"
È il fenomeno per cui i gradienti diventano sempre più piccoli scendendo verso i layer iniziali della rete, fino a impedirne l'apprendimento. Cause: funzioni di attivazione che saturano (sigmoid, tanh) e cattiva inizializzazione dei pesi. Si previene usando ReLU/Leaky ReLU (che non saturano per input positivi), inizializzazione di He o Xavier (che mantiene la varianza dei segnali costante attraverso i layer), e gradient clipping (protegge dal caso opposto, exploding gradient).

9.3 Su algoritmi genetici

Domanda tipo orale

"Spiegami il ciclo di un algoritmo genetico."
Si parte da una popolazione casuale di soluzioni candidate. A ogni generazione: (1) si calcola la fitness di ogni individuo; (2) si selezionano i genitori (tipicamente con torneo); (3) i genitori si combinano via crossover producendo dei figli; (4) i figli vengono mutati con probabilità p_m; (5) la nuova popolazione sostituisce la vecchia, eventualmente preservando il migliore con elitismo. Si itera finché la fitness converge o si raggiunge un numero massimo di generazioni.

Domanda tipo orale

"Cos'è l'elitismo e perché è utile?"
È la tecnica di copiare il migliore individuo della generazione corrente nella nuova senza farlo passare per crossover/mutazione. Garantisce che la fitness massima non possa peggiorare da una generazione all'altra: quello che si è trovato non si perde più. Il costo è una piccola riduzione di diversità (uno slot in meno per nuovi individui), ma con popolazioni di 20+ è trascurabile.

Domanda tipo orale

"Esplorazione vs sfruttamento — come si bilanciano nel tuo GA?"
Diversi parametri controllano questo trade-off. Il `mutation_rate` (0.2) inietta abbastanza casualità da evitare ottimi locali ma non abbastanza da degradare il GA in ricerca casuale. Il `tournament_size` (5) è abbastanza piccolo da dare chance anche agli individui mediocri (esplorazione) ma abbastanza grande da far emergere i migliori (sfruttamento). L'*elitismo* aggiunge sfruttamento puro. La *popolazione* di 20 individui è abbastanza ampia da mantenere diversità senza diluire la pressione selettiva.

Domanda tipo orale

"Cosa succede se aumenti molto il tournament size?"
Aumenta la pressione selettiva: i migliori vincono quasi sempre i tornei, gli individui mediocri quasi mai. Il GA converge più rapidamente ma su un ottimo (anche locale): la popolazione perde diversità in fretta e il GA smette di esplorare. È l'opposto della ricerca casuale — una "hill climbing su steroidi".

9.4 Su NAS e fitness

Domanda tipo orale

"Perché $K = 3$ valutazioni per ogni individuo?"
Perché i pesi della rete sono inizializzati casualmente a ogni valutazione, e con SGD a campioni casuali il training stesso è stocastico. Conseguenza: la stessa architettura, valutata due volte di seguito, può dare accuracy diverse di 1–2% per pura fortuna. Mediare su $K = 3$ run dimezza la deviazione standard a costo ragionevole. Aumentare K riduce ulteriormente la varianza ma con rendimenti decrescenti rapidi.

Domanda tipo orale

"Perché penalizzi la complessità della rete? Cosa succederebbe senza?"
Senza penalità il GA tende a far crescere le architetture, perché reti più grandi hanno tipicamente accuracy leggermente superiore sul validation set (più capacità di adattamento). Questo causa due problemi: (1) overfitting sul validation, con gap val/test elevato; (2) reti inutilmente costose. La penalità $\lambda \cdot \log_{10}(N_{\text{params}})$ agisce come regolarizzatore: scoraggia la complessità non giustificata da un guadagno sostanziale in accuracy.

Domanda tipo orale

"Perché la penalità è logaritmica e non lineare?"
Con la penalità lineare $\lambda \cdot N_{\text{params}}$ il valore ottimale di λ dipende fortemente dalla scala delle reti generate in quella esecuzione: una stessa λ non penalizza praticamente nulla una rete piccola e annienta una rete grande. Con $\log_{10}$ la penalità cresce in modo molto più mite (passare da 100 a 10000 parametri aumenta il termine solo di 2), rendendo λ stabile indipendentemente dalla scala.

Domanda tipo orale

"Come fai a confrontare reti con architetture diverse?"
Tramite la fitness. Ogni rete viene valutata con la stessa procedura: costruita con i suoi parametri di architettura, addestrata con backpropagation per E epoche, valutata in accuracy sul validation set. La fitness è poi accuracy meno la penalità $\lambda \cdot \log_{10}(N_{\text{params}})$, che permette di confrontare reti di dimensioni diverse: una rete piccola con accuracy del 90% può avere fitness più alta di una rete grande con accuracy del 92%.

9.5 Su validazione e leakage

Domanda tipo orale

"Differenza tra validation set e test set?"
Il *validation* si usa *durante* la ricerca, per scegliere fra configurazioni alternative. Il *test* si usa *una sola volta* alla fine, per stimare onestamente le prestazioni. Se si usa il test set durante la ricerca, si introduce data leakage e la stima diventa ottimistica. Nel mio progetto: il GA misura la fitness sul validation, e solo l'architettura vincitrice viene rivalutata sul test set, da cui la metrica finale.

Domanda tipo orale

"Cos'è il data leakage e come lo eviti?"
È qualunque situazione in cui informazione del validation/test "filtra" nel processo di training, gonfiando le stime di performance. Le forme più tipiche sono: (1) calcolare statistiche di preprocessing (es. StandardScaler) sull'intero dataset invece che solo sul training; (2) selezionare un modello con il validation e poi riportare la sua accuracy sul validation come risultato (bias ottimistico). Lo evito calcolando ogni statistica solo sul training, applicandola identica a val/test, e riportando come risultato finale solo la `test_accuracy`.

Domanda tipo orale

"Cosa hai trovato col Test 9A sul bias val/test?"
Su 10 seed diversi, la val accuracy media è del 91.33% ma la test accuracy media è solo 87.01%: un gap sistematico di 4.33%. Questo conferma che la metrica usata per scegliere (validation) non è una stima onesta delle prestazioni: è ottimistica per costruzione, perché il GA ottimizza per quella metrica specifica.

9.6 Domande di approfondimento

Domanda tipo orale

"Il GA potrebbe far evolvere anche i pesi della rete?"
Tecnicamente sì, ma sarebbe enormemente inefficiente. Lo spazio dei pesi è continuo e ad altissima dimensionalità (decine di migliaia di parametri reali). Il GA è efficiente su spazi discreti e di dimensione moderata, mentre la backpropagation, che usa l'informazione del gradiente, è ordini di grandezza più rapida sui pesi. La filosofia del progetto è usare lo stru-

mento giusto per ciascun livello: GA sui parametri discreti (architettura), backpropagation sui pesi continui.

Domanda tipo orale

"Come potresti migliorare il sistema?"
Diverse strade. Estendere il cromosoma per includere learning rate ed epochs (oggi fissi). Crossover "intelligenti" che scambiano layer simili in posizione simile. Fitness multi-obiettivo (NSGA-II) per esplorare il fronte di Pareto fra accuracy e complessità. Surrogate models per stimare la fitness senza training completo, abbattendo il costo. Estendere lo spazio di ricerca a topologie non sequenziali (skip connections).

Domanda tipo orale

"Il tuo GA è collegato alla ricerca locale del libro Russell-Norvig?"
Sì. Il GA può essere visto come una variante della ricerca locale stocastica. Hill climbing tiene un singolo stato e va al successore migliore. Simulated annealing aggiunge tolleranza alle mosse peggiorative. Local beam search tiene k stati in parallelo. Il GA estende il beam search aggiungendo il *crossover*: i nuovi candidati possono combinare informazioni da più stati genitori, non solo essere successori di uno solo. Aggiunge anche la mutazione, che è analoga al rumore casuale di simulated annealing.

Domanda tipo orale

"Ti aspettavi che il GA trovasse architetture con un solo layer?"
A priori no. Mi aspettavo architetture più profonde, soprattutto su Digits che ha 64 feature e 10 classi. Il GA ha invece dimostrato che per questi dataset *tabulari* un singolo layer abbastanza ampio è già sufficiente: l'extra profondità non aggiunge accuracy ma aggiunge parametri, quindi viene scoraggiata dalla penalità λ. Questo è coerente con la letteratura: la profondità è davvero utile quando i dati hanno struttura gerarchica (immagini, linguaggio), non altrettanto sui dataset tabulari semplici.

A Glossario dei termini chiave

Accuracy Percentuale di predizioni corrette sul set di valutazione. Compresa fra 0 e 1 (o 0 e 100%).

Architettura Struttura della rete neurale: numero di layer, neuroni per layer, funzioni di attivazione. È un iperparametro. Nel progetto è il cromosoma del GA.

Backpropagation Algoritmo per calcolare il gradiente della loss rispetto ai pesi della rete, applicando la regola della catena layer per layer all'indietro.

Baseline Modello di riferimento (qui: rete con architettura fissa $[(8, \text{leaky_relu}), (8, \text{leaky_relu})]$) usato per capire se il GA sta aggiungendo valore.

Bias (di un neurone) Termine costante b aggiunto alla somma pesata degli input prima dell'attivazione: $f(\boldsymbol{w}^\top \boldsymbol{x} + b)$.

Bias ottimistico val/test Sovrastima sistematica delle prestazioni quando si riporta come risultato la stessa metrica usata per selezionare il modello.

Classificazione Problema in cui y è una categoria discreta. Metrica naturale: accuracy. Funzione di output: softmax.

Cromosoma Rappresentazione di una soluzione candidata nel GA. Nel progetto: lista di tuple `(n_neuroni, funzione)`.

Cross-entropy Loss function per classificazione. La forma categoriale per multiclasse: $-\sum_i y_i \log \hat{y}_i$.

Crossover Operatore genetico che combina due genitori per generare un figlio.

Data leakage Uso involontario di informazione del validation o test set durante il training, che gonfia le stime di performance.

Elitismo Tecnica di copiare il miglior individuo nella generazione successiva senza farlo subire crossover/mutazione.

Esplorazione Capacità del GA di visitare zone nuove dello spazio delle soluzioni. Promossa da mutazione, popolazione grande, tournament size piccolo.

Fitness Misura di qualità di un individuo nel GA. Nel progetto: accuracy $- \lambda \cdot \log_{10}(N_{\text{params}})$.

Forward pass Calcolo dell'output di una rete a partire da un input.

Gene Elemento atomico di un cromosoma. Nel progetto: una tupla `(n_neuroni, funzione)`.

Generazione Una iterazione completa del ciclo evolutivo del GA.

Gradient descent Algoritmo di ottimizzazione che muove i parametri nella direzione opposta al gradiente della loss.

Iperparametri Parametri di un modello che non vengono appresi dai dati ma fissati prima del training (learning rate, λ, ecc.). L'architettura è un iperparametro.

Learning rate (η) Iperparametro che controlla la dimensione del passo di gradient descent. Il più critico nel progetto.

Loss Funzione che misura quanto la predizione si discosta dalla verità. La backpropagation cerca di minimizzarla.

Mutazione Operatore genetico che introduce variazioni casuali in un cromosoma per mantenere diversità.

NAS (Neural Architecture Search) Sottocampo dell'AutoML che automatizza la progettazione di architetture neurali.

One-hot encoding Rappresentazione delle etichette di classe come vettori binari con un solo 1 nella posizione della classe.

Overfitting Fenomeno per cui un modello memorizza il training invece di generalizzare. Si manifesta con accuracy alta sul training ma bassa sul validation/test.

Parametri (di un modello) Valori che il modello impara dai dati: nel caso di una rete neurale, pesi e bias.

Penalità di complessità Termine sottratto alla fitness per scoraggiare reti grandi. Nel progetto: $\lambda \cdot \log_{10}(N_{\text{params}})$.

Popolazione Insieme di tutti gli individui in una generazione.

ReLU $f(x) = \max(0, x)$. La funzione di attivazione più usata nei layer nascosti.

Selezione a torneo Procedura di selezione: si estraggono k individui casuali e si sceglie il migliore tra loro.

Softmax Funzione di output per classificazione multiclasse: trasforma un vettore in una distribuzione di probabilità.

Sfruttamento Capacità del GA di concentrarsi sulle zone già promettenti. Promosso da elitismo, tournament size grande, basso mutation rate.

StandardScaler Trasformazione che porta ogni feature a media 0 e deviazione standard 1. Da fittare *solo* sul training.

Stratificazione Tecnica di split che preserva le proporzioni delle classi in ciascun sottoinsieme.

Test set Sottoinsieme dei dati toccato *una sola volta* alla fine, per stimare onestamente la performance del modello.

Training set Sottoinsieme dei dati usato per aggiornare i pesi via backpropagation.

Validation set Sottoinsieme usato per scegliere fra configurazioni alternative (qui: per misurare la fitness del GA).